Phase 1(1a & 1b)

```
0703513 (origin/main, origin/HEAD) Initial commit
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ ./test_objects || true
w) ---'
find .pes/objects -type f | sed -n '1,200p'
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ echo '--- objects files (few) ---'
--- objects files (few) ---
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ find .pes/objects -type f | sed -n '1,200p'
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

```
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ ./test_objects 2>&1 | tee out_s01_test_objects.txt
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ find .pes/objects -type f | sort | tee out_s02_objects
_shard.txt
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$
```

Phase 2(2a & 2b)

```
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ ./test_tree 2>&1 | tee out_s03_test_tree.txt
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ for f in $(find .pes/objects -type f | sort); do
  if head -c 5 "$f" | grep -q '^tree'; then
    echo "xxd of $f (first 20 lines):"; xxd "$f" | sed -n '1,20p' > out_s04_tree_xxd.txt
    cat out_s04_tree_xxd.txt
    break
  fi
done
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$
```

Phase 3(3a & 3b)

```
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ ./pes init 2>&1 | tee out_s05_pes_init.txt
./pes add file1.txt 2>&1 | tee out_s05_pes_add.txt
./pes status 2>&1 | tee out_s05_pes_status.txt
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  TEMP_PHASE4_A.txt
  untracked:  out_phase4_pes_commit2.txt
  untracked:  out_s03_objects_list.txt
  untracked:  PHASE2_STATUS.txt
  untracked:  tree.c
  untracked:  index.h
  untracked:  out_phase4_retry2_commit3.txt
  untracked:  out_phase4_retry2_commit1.txt
  untracked:  out_phase4_retry_commit1.txt
  untracked:  commit.h
  untracked:  out_s08_pes_files.txt
  untracked:  PHASE4_STATUS.txt
  untracked:  out_s05_pes_init.txt
  untracked:  index.c
  untracked:  out_s01_test_objects.txt
  untracked:  PHASE4_SCREENSHOTS.txt
  untracked:  out_phase4_retry_commit2.txt
  untracked:  PHASE3_SCREENSHOTS.txt
  untracked:  PHASE3_STATUS.txt
```

```
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ cat .pes/index | tee out_s06_index.txt
100664 8c305f43b012ca0752a1ec1165954d4bfd6650cf25a15496d85072c56f8ec6c4 1776735301 26 file1.txt
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$
```

4(A & B&C)

```
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ sed -n '1,200p' out_s07_pes_log.txt
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ sed -n '1,200p' out_s08_pes_files.txt
.pes/HEAD
.pes/index
.pes/objects/09/edaadb2431ea43cb7bc5f016a5358d1b39b3b3985cade8b7de9979bc4dee47
.pes/objects/27/84388ca138b0a432dbd5c16120cfe999278a53d4765726feda84b1ee807fe9
.pes/objects/46/c5a2ecd6b7ba11101f514086c589d05ae647f499a24b0684c7ade71316c8d3
.pes/objects/8d/749528c828c14f2498781a3f519265c148622ad66fa032033a28e7c8d8d841
.pes/objects/92/9ee053f582d27f8bfc95d006ae10e660e1115e64cead62986bca8ac821ca2d
.pes/objects/a1/d53e47579eb7e409a9c540ce075acff152ea2c8227ae64359db6e4284c515c
.pes/refs/heads/main
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ sed -n '1,200p' out_s09_ref_main.txt
sed -n '1,200p' out_s09_HEAD.txt
2784388ca138b0a432dbd5c16120cfe999278a53d4765726feda84b1ee807fe9
ref: refs/heads/main
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$
```

Final ()

```
=== All integration tests completed ===
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$ make test-integration 2>&1 | tee out_final_integratio
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
commit: no parent
commit: serialized 174 bytes
commit: wrote object 43c20734feeacaf0d078c27eb70f1bd896085eea2f13b61a72abb86848ebb0b5
commit: head updated
Committed: 43c20734feea... Initial commit

Log after first commit:
commit 43c20734feeacaf0d078c27eb70f1bd896085eea2f13b61a72abb86848ebb0b5
Author: PES User <pes@localhost>
Date:   1776737213

    Initial commit


--- Second Commit ---
commit: found parent 43c20734feeacaf0d078c27eb70f1bd896085eea2f13b61a72abb86848ebb0b5
commit: serialized 247 bytes
commit: wrote object 0d48edae5831efdbc3bc338dc4d088a1aff42df341bddefc214f6d6265f58740
commit: head updated
Committed: 0d48edae5831... Update file.txt

--- Third Commit ---
commit: found parent 0d48edae5831efdbc3bc338dc4d088a1aff42df341bddefc214f6d6265f58740
commit: serialized 244 bytes
commit: wrote object b8a79151ff955257abc36e9033bf3c708fab1988ecd3f1abe06cbe8c18bd2338
commit: head updated
Committed: b8a79151ff95... Add farewell

--- Full History ---
commit b8a79151ff955257abc36e9033bf3c708fab1988ecd3f1abe06cbe8c18bd2338
Author: PES User <pes@localhost>
```

```
Author: PES User <pes@localhost>
Date:   1776737213

    Add farewell

commit 0d48edae5831efdbc3bc338dc4d088a1aff42df341bddefc214f6d6265f58740
Author: PES User <pes@localhost>
Date:   1776737213

    Update file.txt

commit 43c20734feeacaf0d078c27eb70f1bd896085eea2f13b61a72abb86848ebb0b5
Author: PES User <pes@localhost>
Date:   1776737213

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
b8a79151ff955257abc36e9033bf3c708fab1988ecd3f1abe06cbe8c18bd2338

--- Object Store ---
Objects created:
6
.pes/objects/0d/48edae5831efdbc3bc338dc4d088a1aff42df341bddefc214f6d6265f58740
.pes/objects/43/c20734feeacaf0d078c27eb70f1bd896085eea2f13b61a72abb86848ebb0b5
.pes/objects/b8/a79151ff955257abc36e9033bf3c708fab1988ecd3f1abe06cbe8c18bd2338
.pes/objects/d3/648385320928d3b93bdb418f1c54893e470a4b6289e91374f7290ea4b11168
.pes/objects/f3/54f31b93f02738bd9b32cfb4141f73e374bef04d0c6a93a8b6af6a6b1a6ed1
.pes/objects/fe/71f64cc98674d765ae24e70f243b39527eb27334936aa460f93988ba892b13

=== All integration tests completed ===
ved-m@ved-m-QEMU-Virtual-Machine:~/PESXUG24CS522-pes-vcs$
192.168.64.2   main*  0↓ 47↑   ⊗ 0 ⚠ 0   0
```

Phase (5 & 6)

Branching and Checkout

Q5.1: Implementing pes checkout <branch>

.pes/ changes: The .pes/HEAD file must be updated to point to the new branch reference (e.g., ref: refs/heads/<branch>) instead of the old one. The index must also be updated to match the tree of the target branch.

Working directory changes: Files must be updated to match the exact state of the target branch's commit tree. This means deleting files that don't exist in the new branch, creating missing ones, and overwriting modified ones.

Complexity: The hardest part is making sure you don't overwrite or lose the user's uncommitted local changes while swapping out the rest of the file system.

Q5.2: Detecting a "dirty working directory" conflict

First, compare the working directory files to the current index. If a file's hash in the working directory differs from its hash in the index, the file is "dirty" (uncommitted changes).

Next, look up that specific file in the object store for both the current branch's tree and the target branch's tree.

If the file is dirty AND its stored hash differs between the current and target branches, checkout must abort to prevent overwriting the user's unsaved work.

Q5.3: Detached HEAD commits and recovery

What happens: When you make a commit in a detached HEAD state, the new commit object is created and .pes/HEAD is updated with the new commit hash, but no branch pointer (refs/heads/...) moves forward to track it.

The risk: If you checkout a different branch, those commits become "orphaned" or unreachable because nothing points to them.

Recovery: A user can recover them by creating a new branch that explicitly points to the orphaned commit's hash (e.g., git branch rescue-branch <hash>). The hash can usually be found in terminal history or a reference log (if your VCS tracks one).

Garbage Collection and Space Reclamation

Q6.1: Finding and deleting unreachable objects

Algorithm: A classic "Mark and Sweep" algorithm.

Mark: Start at all known branch references (the roots). Traverse every commit they point to, parse those commits to find their trees, and parse the trees to find sub-trees and blobs. Mark every hash you encounter as "reachable".

Sweep: Iterate through every file inside .pes/objects/. If a file's hash is not in your marked list, delete it.

Data Structure: A Hash Set (or a Bloom Filter if memory is strictly constrained) is best to track "reachable" hashes efficiently with $O(1)$ lookups.

Estimate: For 100,000 commits, you will visit 100,000 commit objects and at least 100,000 root tree objects. Depending on how many files change per commit, you will visit hundreds of thousands of additional blob and sub-tree objects. At a bare minimum, you are visiting 200,000+ objects.

Q6.2: Concurrent Garbage Collection dangers

The Race Condition: Imagine a user runs pes commit. The system creates a new blob and tree in .pes/objects/, but hasn't yet written the final commit object or updated the branch pointer. At that exact millisecond, the background GC runs. The GC scans the branches, doesn't see the brand-new blob/tree (because the branch pointer hasn't moved yet), assumes they are "unreachable", and deletes them.

The commit finishes, resulting in a corrupted repository with missing objects.

Git's Real Solution: Git avoids this by using a "grace period". Its garbage collection only deletes unreachable objects that are older than a certain threshold (usually 2 weeks). New, pending objects are completely ignored by the GC.